

Análise do Impacto de Variações de Prompting e Feedback Semântico no ARC-AGI-1: Fundamentação Estatística

Fabio Luiz da Costa Cieslak^{*1}, Izadora Candotti de Oliveira^{*1}, Matheus Machado^{*1}

Universidade Federal do Rio Grande do Sul¹

Abstract

O benchmark Abstraction and Reasoning Corpus (ARC-AGI-1) avalia o raciocínio indutivo e a generalização em inteligência artificial. Investigamos como variações na estrutura de prompts afetam o desempenho de modelos de linguagem (LLMs) na síntese de programas Python. Comparamos a geração direta (*1-Shot*) à síntese indutiva guiada por contraexemplos (CEGIS), aplicando restrições sintáticas e eliciação de invariantes abstratos. Formalizamos um protocolo estatístico rigoroso com teste exato de McNemar, bootstrap pareado (intervalos de confiança de 95%) e uma taxonomia de falhas (recuperação semântica, regressão, sobreajuste espúrio e teto de representação). Esse arcabouço permite mensurar ganhos reais de desempenho frente ao ruído estocástico dos modelos.

Código e Dados — <https://github.com/Jubarte27/arc-agi-1>

1 Introdução

O Abstraction and Reasoning Corpus (ARC-AGI) (Chollet 2019) avalia a aquisição de habilidades e o raciocínio indutivo amplo, priorizando a generalização sobre a memorização. Cada tarefa é composta por 2 a 5 pares de matrizes de treino e grades de teste cujas saídas devem ser inferidas.

Uma abordagem comum converte o problema em síntese de programas (*Program Synthesis*) (Solar-Lezama 2008), instruindo LLMs a gerar a função `transform(grid)`. O desempenho dessa abordagem depende da estrutura do prompt (Wei et al. 2022), de restrições no espaço de busca e de mecanismos de correção via execução.

Este trabalho investiga o impacto de variações estruturadas de prompt no ARC-AGI-1, contrastando a geração direta com ciclos iterativos de feedback semântico (CEGIS). Além disso, formalizamos uma metodologia estatística pareada para guiar experimentos no benchmark e mitigar falsos positivos decorrentes da variabilidade amostral.

2 Variações de Prompting e Síntese de Programas

O arcabouço experimental compara variações estruturadas de prompts projetadas para induzir diferentes regimes de

busca e raciocínio no LLM.

2.1 Síntese Direta 1-Shot (Baseline)

No *baseline* (1-Shot), o modelo recebe a tarefa e as demonstrações de treino em formato JSON em uma única requisição, produzindo apenas o bloco da função `transform(grid)`.

O prompt inclui regras de restrição de bibliotecas (ANTI_IMPORT_RULES):

- Proibição de importações externas (`import numpy, collections, math, etc.`);
- Uso exclusivo de tipos e funções nativas (`list, dict, range, enumerate, zip, min, max`).

Essas regras eliminam divergências de ambiente e induzem o modelo a explicitar as operações matriciais em código puro e determinístico.

2.2 Feedback Semântico via Contraexemplos (CEGIS)

Na abordagem CEGIS (*Counterexample-Guided Inductive Synthesis*), o prompt inicial expande-se em um diálogo iterativo orientado pelo resultado da execução em sandbox isolada.

Caso o código falhe em qualquer demonstração de treino, o sistema isola o primeiro contraexemplo discordante e constrói um prompt de retorno com:

1. Índice da demonstração violada;
2. Grade de entrada (*Input*);
3. Grade esperada de referência (*Expected Output*);
4. Grade produzida (*Produced Output*) ou erro de execução (*Execution Error*).

O refinamento prossegue iterativamente até a convergência em treino ou o limite de $K_{\max} = 5$ iterações.

2.3 Regras Anti-Trapça e Elicitação de Invariantes

Ao receber feedback de erro, LLMs frequentemente introduzem ramificações condicionais ad-hoc (*if/else*) restritas aos contraexemplos, gerando sobreajuste aos dados visíveis.

Para mitigar esse comportamento, o prompt incorpora regras anti-trapça (ANTI_CHEAT_RULES):

^{*}These authors contributed equally.

- Proibição explícita de condições baseadas em índices de treino ou coordenadas pontuais;
- Elicitação de invariantes: o modelo deve descrever em linguagem natural, antes do código, a regra abstrata unificada que corrige a falha anterior e atende a todas as demonstrações.

3 Metodologia e Fundamentação Estatística

Adotamos testes de hipóteses pareados não-paramétricos para avaliar formalmente as estratégias de prompting.

3.1 Formulação das Hipóteses

Sejam M_{base} e M_{var} duas estratégias de prompting. Para cada tarefa $i \in \{1, \dots, N\}$, a variável $X_i \in \{0, 1\}$ indica sucesso estrito (acerto conjunto em treino e teste). Formulamos dois testes:

1. Diferença Geral ($H_{0,\text{dif}}$):

$$H_{0,\text{dif}} : P(X_i^{(M_{\text{var}})} = 1) = P(X_i^{(M_{\text{base}})} = 1) \quad (1)$$

com alternativa bilateral $H_{1,\text{dif}} (P(X_i^{(M_{\text{var}})} = 1) \neq P(X_i^{(M_{\text{base}})} = 1))$.

2. Superioridade Direcional ($H_{0,\text{melhor}}$):

$$H_{0,\text{melhor}} : P(X_i^{(M_{\text{var}})} = 1) \leq P(X_i^{(M_{\text{base}})} = 1) \quad (2)$$

com alternativa unilateral $H_{1,\text{melhor}} (\Delta > 0)$.

3.2 Desenho Pareado e Tabela de Contingência

Como ambos os métodos avaliam o mesmo conjunto de tarefas, os resultados formam uma matriz 2×2 :

- $a = n_{11}$: acerto em ambos os métodos;
- $b = n_{10}$: acerto exclusivo do Baseline (regressões);
- $c = n_{01}$: acerto exclusivo do CEGIS (recuperações);
- $d = n_{00}$: falha em ambos os métodos.

3.3 Teste Exato de McNemar

Para testar $H_{0,\text{dif}}$, aplicamos o teste exato de McNemar (McNemar 1947). Sob a nula, os pares discordantes ($n_{\text{disc}} = b + c$) seguem uma distribuição Binomial($b + c, 0.5$). O p -valor bilateral é:

$$p_{\text{McNemar}} = \min \left(1, 2 \sum_{k=0}^{\min(b,c)} \binom{b+c}{k} \left(\frac{1}{2} \right)^{b+c} \right) \quad (3)$$

A formulação exata é essencial no ARC, onde o baixo número de casos discordantes invalida a aproximação assintótica por χ^2 .

3.4 Inferência por Bootstrap Pareado

Para avaliar $H_{0,\text{melhor}}$ e estimar a incerteza do efeito, empregamos bootstrap pareado ($B = 10.000$ reamostragens com reposição) (Efron and Tibshirani 1993). Em cada iteração j :

$$\Delta_j^* = \frac{1}{N} \sum_{i=1}^N (X_{i,j}^{*(M_{\text{var}})} - X_{i,j}^{*(M_{\text{base}})}) \quad (4)$$

Com a diferença observada $\hat{\Delta} = (c - b)/N$, centralizamos as réplicas sob H_0 ($\tilde{\Delta}_j^* = \Delta_j^* - \hat{\Delta}$). O p -valor empírico é:

$$p_{\text{boot}} = \frac{1 + \sum_{j=1}^B \mathbb{I}(\tilde{\Delta}_j^* \geq \hat{\Delta})}{B + 1} \quad (5)$$

onde $\mathbb{I}(\cdot)$ é a função indicadora. O intervalo de confiança de 95% é obtido a partir dos percentis 2.5% e 97.5% de Δ^* .

3.5 Taxonomia de Falhas e Transições de Estado

decompõe-se o comportamento das soluções em quatro categorias operacionais:

1. **Recuperação Semântica** ($0 \rightarrow 1$): Falha no Baseline revertida com sucesso no teste cego via CEGIS.
2. **Regressão** ($1 \rightarrow 0$): Acerto no Baseline perdido durante o refinamento iterativo.
3. **Sobreajuste Espúrio / Falsa Convergência**: Acerto total em treino seguido de falha no teste cego.
4. **Teto de Representação**: Falha em convergir no treino após K_{max} iterações.

4 Protocolo Experimental e Execução

A infraestrutura garante reprodutibilidade e conformidade com limites operacionais das APIs.

4.1 Critério de Avaliação e Sandbox

Os programas são executados em sandbox isolada com limite de tempo (`TIMEOUT_SECONDS = 2.0s`). O acerto exige que a função `transform(grid)` produza matrizes estritamente idênticas às de referência no treino e no teste cego.

4.2 Modelos e Infraestrutura de Execução

O pipeline suporta modelos comerciais (ex.: `gemin-3.1-flash-lite`) e de pesos abertos (ex.: `llama-3.3-70b-versatile` via Groq ou Ollama com ROCm/CUDA).

Mecanismos de estabilidade e controle incluem:

- Controle de taxa de requisições por minuto (RPM) e tokens por minuto (TPM);
- Monitoramento de cota diária (RPD);
- Persistência incremental (*Checkpoint & Resume*) após cada tarefa individual concluída.

5 Discussão e Escalonamento Empírico

O feedback por contraexemplos introduz um compromisso estrutural: embora viabilize a correção de hipóteses geométricas incorretas, amplia o risco de sobreajuste em tarefas com poucos exemplos.

A metodologia proposta quantifica esse efeito com rigor. No ARC-AGI-1 completo (400 tarefas de treino e 400 de avaliação), o protocolo assegura poder estatístico ($1 - \beta$) suficiente para detectar diferenças com nível de significância $\alpha = 0.05$, mesmo diante de ganhos modestos de acurácia.

6 Conclusão

Este trabalho apresenta uma metodologia formal para avaliar o impacto de variações de prompt e feedback semântico no ARC-AGI-1. A integração entre prompts estruturados e inferência estatística pareada (McNemar e bootstrap) estabelece uma base rigorosa e reproduzível para a síntese de programas com LLMs.

Agradecimentos

Agradecemos aos colaboradores do projeto `arc-agi-1` pelo desenvolvimento dos módulos de execução e análise estatística.

References

- Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.
- Efron, B.; and Tibshirani, R. J. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.
- McNemar, Q. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2): 153–157.
- Solar-Lezama, A. 2008. *Program Synthesis by Sketching*. Ph.D. thesis, University of California, Berkeley.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35: 24824–24837.